

AUTONOMOUS PARKING VEHICLE

Using ESP32, Ultrasonic Sensors and 8-bit IR Array

PROJECT REPORT

IoT | Embedded Systems | Robotics | Autonomous Vehicles

ABSTRACT

This project presents a compact autonomous parking vehicle based on an ESP32 microcontroller. The system demonstrates sensor-based environment perception, obstacle detection, boundary detection, vehicle alignment and automated reverse parking. Four HC-SR04 ultrasonic sensors are mounted at the front, rear, left and right sides to measure surrounding distances. An 8-bit digital IR sensor array is used for boundary or line detection. The ESP32 processes sensor information and controls four DC motors through an L298N motor driver. A finite-state-machine controller divides operation into APPROACH, ALIGN, POSITION, REVERSE PARKING, PARKED and EMERGENCY STOP states. A MATLAB simulation was developed as a preliminary validation of the autonomous parking concept before hardware implementation.

Keywords: ESP32, Autonomous Parking, HC-SR04, IR Sensor Array, L298N, Embedded Systems, Robotics, IoT, Obstacle Detection, State Machine

1. INTRODUCTION

Autonomous parking is an important application of intelligent transportation and robotic systems. A parking vehicle must sense its surroundings, detect obstacles, estimate its relative position and execute controlled motion without continuous manual intervention. This project demonstrates these concepts using a low-cost ESP32-based prototype. Four ultrasonic sensors provide directional distance information while an 8-bit IR array provides floor-level boundary or line information. The ESP32 converts these measurements into motor commands.

The concept was first developed in MATLAB using a vehicle, parking slot, obstacles, virtual sensors and a state-machine controller. The hardware implementation transfers the core sensing and decision-making concept to an embedded platform.

2. PROBLEM STATEMENT

Manual parking can be difficult in constrained spaces because the driver must continuously judge the vehicle's distance from surrounding objects. The objective of this project is to develop a small autonomous vehicle that can detect obstacles and boundaries and perform a controlled parking sequence using an ESP32.

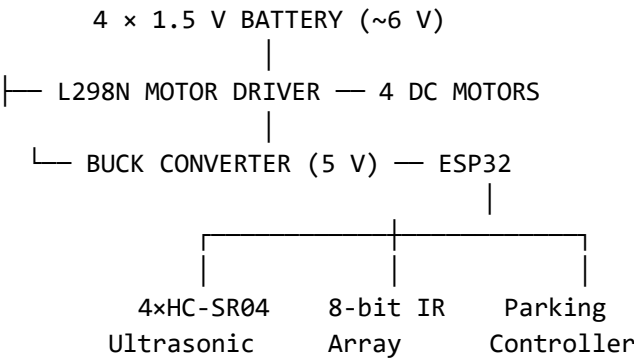
3. OBJECTIVES

- Develop an ESP32-based autonomous parking prototype.
- Measure front, rear, left and right distances using four HC-SR04 sensors.
- Use an 8-bit IR sensor array for boundary/line detection.
- Control four DC motors through an L298N motor driver.
- Implement obstacle-based emergency stopping.
- Implement vehicle alignment and positioning logic.
- Perform automated reverse parking.
- Validate the concept through MATLAB simulation before hardware testing.

4. PROPOSED SYSTEM

The vehicle uses four-wheel differential drive. The two left motors operate together and the two right motors operate together. Differential direction allows forward, reverse, left and right movement. The front ultrasonic sensor is primarily used during approach, the rear sensor during reverse parking, and the side sensors assist alignment and positioning. The IR array provides additional boundary information.

5. SYSTEM ARCHITECTURE



6. HARDWARE REQUIREMENTS

Component	Qty.	Purpose
ESP32 Development Board	1	Main controller
HC-SR04 Ultrasonic Sensor	4	Directional distance/obstacle detection
8-bit Digital IR Sensor Array	1	Boundary/line detection
74HC165 Shift Register	1	Reads eight IR digital outputs using three ESP32 GPIOs
L298N Motor Driver	1	DC motor direction and speed control
DC Motors	4	Vehicle propulsion
Buck Converter	1	Regulated 5 V supply
1.5 V Battery Cells	4	Approximately 6 V nominal in series
1 kΩ Resistors	4	Upper resistor for ECHO voltage dividers
2 kΩ Resistors	4	Lower resistor for ECHO voltage dividers
Chassis, wheels and jumper wires	As required	Mechanical/electrical assembly

7. SOFTWARE REQUIREMENTS

- Arduino IDE with ESP32 board support.
- C/C++ firmware for ESP32.
- Serial Monitor at 115200 baud for debugging.
- MATLAB for preliminary simulation and controller development.

8. CIRCUIT AND PIN CONFIGURATION

8.1 Ultrasonic Sensors

Sensor	Position	TRIG	ECHO
HC-SR04 #1	Front	GPIO 25	GPIO 34
HC-SR04 #2	Rear	GPIO 26	GPIO 35
HC-SR04 #3	Left	GPIO 27	GPIO 32
HC-SR04 #4	Right	GPIO 14	GPIO 33

All ultrasonic VCC connections use the 5 V supply and all grounds are common. When powered at 5 V, each HC-SR04 ECHO line should be reduced to an ESP32-safe level using a voltage divider.

Recommended ECHO divider: HC-SR04 ECHO → 1 kΩ → ESP32 ECHO input, with 2 kΩ from the ESP32 input node to GND.

8.2 L298N Motor Driver

L298N	ESP32
ENA	GPIO 4
IN1	GPIO 16
IN2	GPIO 17
IN3	GPIO 18
IN4	GPIO 19
ENB	GPIO 21

OUT1/OUT2 are assigned to the left motor side and OUT3/OUT4 to the right motor side. Two motors on a driver channel must only be used if the driver and power source can safely handle their combined current.

8.3 8-bit IR Array and 74HC165

IR Array	74HC165
VCC	VCC
GND	GND
D1	D0
D2	D1
D3	D2
D4	D3
D5	D4
D6	D5
D7	D6
D8	D7
74HC165 Signal	ESP32
Q7 / DATA	GPIO 22
CP / CLOCK	GPIO 23
PL / LOAD	GPIO 13

The 74HC165 converts the eight parallel IR states into a serial data stream, reducing the ESP32 GPIO requirement to three pins.

8.4 Power Supply

Four 1.5 V cells are intended to be connected in series, providing approximately 6 V nominal. The battery supplies the motor driver and the buck converter. The buck converter is adjusted to approximately 5 V for the

ESP32 and suitable 5 V sensor supply. Battery negative, motor-driver GND, buck-converter GND, ESP32 GND and sensor grounds must share a common ground.

9. WORKING PRINCIPLE

- Sensing:** Four ultrasonic sensors measure distances around the vehicle, while the IR array detects the configured boundary/line pattern.
- Processing:** The ESP32 reads the sensor values and evaluates the current state.
- Decision:** The controller selects forward, alignment, positioning, reverse, parked or emergency-stop behavior.
- Actuation:** The L298N drives the four DC motors according to the selected command.
- Safety:** If a state-relevant obstacle is inside the emergency distance, the motors are stopped until the area is clear.

10. AUTONOMOUS PARKING ALGORITHM

State	Function	Main sensing
APPROACH	Move toward the parking area.	Front ultrasonic
ALIGN	Correct vehicle orientation/side clearance.	Left/right ultrasonic
POSITION	Center the vehicle in the available space.	Left/right ultrasonic + safety
REVERSE PARKING	Move backward toward the final position.	Rear ultrasonic
PARKED	Stop the vehicle.	Motors stopped
EMERGENCY STOP	Stop immediately when unsafe distance is detected.	Relevant ultrasonic sensors

Initial firmware thresholds are 15 cm for emergency stopping, 35 cm for the safe-distance transition, and 12 cm for the reverse-parking stopping point. Motor speeds are software parameters and should be calibrated for the actual chassis.

11. MATLAB SIMULATION

The autonomous parking concept was developed and tested in MATLAB before hardware implementation. The simulation included a vehicle, parking slot, obstacles, virtual front/rear/left/right distance sensors, vehicle trajectory visualization, a state-machine controller and emergency-stop logic. The simulation served as a development baseline for transferring the sensing and decision-making strategy to the ESP32 prototype.

12. ESP32 IMPLEMENTATION

The ESP32 continuously reads the ultrasonic sensors and the eight IR channels. HC-SR04 distance is calculated from echo pulse duration. The IR array is loaded into the 74HC165 and shifted into the ESP32 as an eight-bit value. The motor driver receives direction signals and PWM speed commands. The Serial Monitor displays directional distance measurements, IR data and the active parking state.

13. TESTING AND EXPECTED OPERATION

- Verify regulated 5 V supply before connecting the ESP32.
- Verify each ultrasonic sensor and then all four directions.
- Verify D1–D8 response from the IR array.
- Test motor directions with the wheels lifted from the ground.
- Verify forward, reverse, left and right movement.
- Place an obstacle in the path and verify emergency stopping.
- Run the complete parking sequence in a controlled test area.
- Calibrate sensor thresholds and motor speeds for the physical chassis.

Expected Serial Monitor Output

F:85.2cm R:91.4cm L:43.8cm Rt:45.1cm IR:11111111
STATE: APPROACH

STATE: ALIGN
STATE: POSITION
STATE: REVERSE PARKING

```
=====
      VEHICLE PARKED
=====
```

14. ADVANTAGES

- Low-cost embedded prototype.
- Four-direction obstacle sensing.
- Dedicated rear sensing for reverse parking.
- Eight-channel IR boundary detection.
- Structured state-machine control.
- MATLAB simulation before hardware implementation.
- Expandable ESP32 platform.

15. LIMITATIONS

- Ultrasonic readings can be affected by surface angle and object shape.
- Parking accuracy depends on sensor mounting, chassis geometry and motor matching.
- L298N has voltage-drop and current limitations.
- The current controller is rule-based rather than a full path-planning/localization system.
- IR behavior depends on sensor threshold and mounting height.
- A small battery pack may not provide sufficient motor current under heavy load.

16. APPLICATIONS

- Smart parking demonstrations.
- Educational robotics.
- Embedded-systems laboratories.

- Indoor autonomous robots.
- Warehouse vehicle concepts.
- Autonomous vehicle research prototypes.

17. FUTURE ENHANCEMENTS

- Camera-based parking-slot recognition.
- Computer vision for parking-space detection.
- PID-based motion control.
- Automatic parking-slot identification.
- Wi-Fi/web dashboard.
- Mobile application and manual override.
- Battery voltage/current monitoring.
- IoT cloud data logging.
- Advanced path planning and localization.

18. CONCLUSION

The Autonomous Parking Vehicle project demonstrates the integration of embedded computing, distance sensing, boundary detection and motor control to perform an automated parking sequence. The ESP32 serves as the central controller, four HC-SR04 sensors provide directional obstacle information, the 8-bit IR array supports boundary detection, and the L298N controls the four-wheel drive system. A state-machine architecture organizes approach, alignment, positioning, reverse parking, parked and emergency-stop behavior. The MATLAB simulation provides a useful development baseline, while the ESP32 implementation provides a practical hardware prototype that can be extended with computer vision, improved localization, path planning and IoT monitoring.

19. REFERENCES

- ESP32 development-board and Arduino ESP32 documentation.
- HC-SR04 ultrasonic sensor interfacing references.
- L298N dual H-bridge motor-driver documentation.
- 74HC165 parallel-in/serial-out shift-register documentation.
- MATLAB documentation for simulation and visualization.

APPENDIX – COMPLETE GPIO SUMMARY

Subsystem	Signal	GPIO
Front HC-SR04	TRIG	25
Front HC-SR04	ECHO	34
Rear HC-SR04	TRIG	26
Rear HC-SR04	ECHO	35
Left HC-SR04	TRIG	27
Left HC-SR04	ECHO	32
Right HC-SR04	TRIG	14

Right HC-SR04	ECHO	33
L298N	ENA	4
L298N	IN1	16
L298N	IN2	17
L298N	IN3	18
L298N	IN4	19
L298N	ENB	21
74HC165	DATA	22
74HC165	CLOCK	23
74HC165	LOAD	13